

數位影像處理 HW2

Canny Edge Detection、Hough Transform

資訊三乙 D1249326 陳卓妍

一、原圖



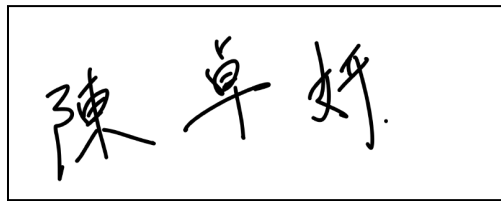
圖一、1. jpg



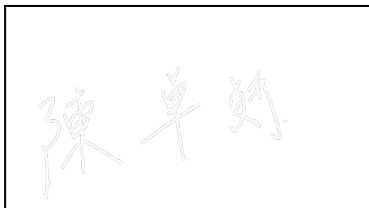
圖二、2. jpg



圖三、3. jpg



圖四、signed. png



圖五、white_signed. png

二、程式碼

```
import cv2
import numpy as np
import math
import os

# 1. 轉灰階
def rgb2gray(img):
    h, w, c = img.shape
    gray = np.zeros((h, w), dtype=np.uint8)

    for y in range(h):
        for x in range(w):
            r, g, b = img[y, x]
            # 轉灰階公式  $gray = 0.299 * R + 0.587 * G + 0.114 * B$ 
            gray[y, x] = int(0.299 * r + 0.587 * g + 0.114 * b)

    return gray

# 2. 高斯模糊
# 建立高斯核  $\Rightarrow$  找到邊緣前，先將影像進行平滑處理，減少雜訊影響
def gaussian_kernel(size, sigma):
    # 計算核心位置
    k = size // 2
    # 建立空矩陣 for 高斯核
    kernel = np.zeros((size, size), dtype=np.float32)
    s = 2 * sigma * sigma

    for i in range(size):
        for j in range(size):
            x = i - k
            y = j - k
            # 計算該位置的高斯權重值
            kernel[i, j] = math.exp(-(x * x + y * y) / s)

    # 正規化
```

```

    return kernel / np.sum(kernel)

# 定義高斯模糊函式
def gaussian_blur(img, size=3, sigma=1.4):
    h, w = img.shape
    kernel = gaussian_kernel(size, sigma)
    # 計算padding大小
    # eg: size=3 ⇒ pad=1, size=5 ⇒ pad=2
    pad = size // 2

    # 進行padding(邊界補植) 用reflect(鏡射)模式 ⇒ 減少邊界效應(黑框)
    padded = np.pad(img, ((pad, pad), (pad, pad)), mode='reflect')
    # 建立輸出影像
    result = np.zeros_like(img, dtype=np.float32)

    # 卷積運算
    # 對影像每個像素進行處理，離核心越近權重越高
    for i in range(h):
        for j in range(w):
            region = padded[i:i+size, j:j+size]
            result[i, j] = np.sum(region * kernel)

    #轉回8-bit 灰階影像
    return result.astype(np.uint8)

# 3. Sobel梯度計算 (計算邊緣強度、方向)
def sobel_gradient(img):
    h, w = img.shape

    # 找垂直邊緣(左右亮度差)
    sobel_x = np.array([[-1, 0, 1],
                        [-2, 0, 2],
                        [-1, 0, 1]], dtype=np.float32)

    # 找水平邊緣(上下亮度差)
    sobel_y = np.array([[-1, -2, -1],
                        [0, 0, 0],

```

```

[1, 2, 1]], dtype=np.float32)

# 進行padding
padded = np.pad(img, ((1, 1), (1, 1)), mode='reflect')

# 建立梯度矩陣
gx = np.zeros((h, w), dtype=np.float32)
gy = np.zeros((h, w), dtype=np.float32)

# 卷積運算
for y in range(h):
    for x in range(w):
        region = padded[y:y+3, x:x+3]
        # 數字越大表示邊緣越強
        gx[y, x] = np.sum(region * sobel_x)
        gy[y, x] = np.sum(region * sobel_y)

# 計算邊緣強度
magnitude = np.sqrt(gx**2 + gy**2)

# 將強度值限制在0-255之間
magnitude = np.clip(magnitude, 0, 255)

# 計算邊緣方向 (0-180度)
direction = np.rad2deg(np.arctan2(gy, gx))
direction[direction < 0] += 180

return magnitude.astype(np.uint8), direction

# 4. 非極大值抑制
# 保留局部最大值
def non_max_suppression(mag, direction):
    h, w = mag.shape
    # 建立非極大值抑制後的影像
    nms = np.zeros((h, w), dtype=np.uint8)

```

```

# 遍歷每個像素，避開邊界
for i in range(1, h - 1):
    for j in range(1, w - 1):
        # 取得該像素的方向
        d = direction[i, j]

        # 方向接近 0度 水平梯度 ⇒ 垂直邊緣
        # 與左右像素比較
        if (0 ≤ d < 22.5) or (157.5 ≤ d ≤ 180):
            p1, p2 = mag[i, j-1], mag[i, j+1]
        # 方向接近 45度 ⇒ 斜向梯度
        # 與左上、右下像素比較
        elif (22.5 ≤ d < 67.5):
            p1, p2 = mag[i-1, j+1], mag[i+1, j-1]
        # 方向接近 90度 垂直梯度 ⇒ 水平邊緣
        # 與上下像素比較
        elif (67.5 ≤ d < 112.5):
            p1, p2 = mag[i-1, j], mag[i+1, j]
        # 方向接近 135度 ⇒ 斜向梯度
        # 與右上、左下像素比較
        else:
            p1, p2 = mag[i-1, j-1], mag[i+1, j+1]

        # (i,j)的梯度值 ≥ 兩側像素值 才保留，否則設為0
        nms[i, j] = mag[i, j] if mag[i, j] ≥ p1 and mag[i, j] ≥
p2 else 0

    return nms

# 5. 雙閾值與邊緣連接
def hysteresis(img, low, high):
    h, w = img.shape
    # 定義強邊緣與弱邊緣的像素值
    strong = 255
    weak = 50

    # 建立輸出影像

```

```

result = np.zeros((h, w), dtype=np.uint8)

# 標記強邊緣(大於high)與弱邊緣(介於兩閾值之間)
strong_i, strong_j = np.where(img ≥ high)
weak_i, weak_j = np.where((img ≤ high) & (img ≥ low))

# 標記強邊緣與弱邊緣
result[strong_i, strong_j] = strong
result[weak_i, weak_j] = weak

# 邊緣追蹤(雙閾值核心)
# 把所有強邊緣座標放入堆疊
stack = list(zip(strong_i, strong_j))

# DFS ⇒ 追蹤連接的弱邊緣
# 只要有強邊緣就要繼續往周圍找弱邊緣
while stack:
    # 從堆疊取出強邊緣座標
    i, j = stack.pop()

    # 檢查8個鄰近像素
    for di in range(-1, 2):
        for dj in range(-1, 2):
            # 跳過自己
            if di == 0 and dj == 0:
                continue
            # 計算鄰近像素座標
            ni, nj = i + di, j + dj

            # 確保鄰近像素在影像範圍內
            if 0 ≤ ni < h and 0 ≤ nj < w:
                # 如果鄰近像素是弱邊緣，則將其標記為強邊緣，並加入堆疊繼續追
                if result[ni, nj] == weak:
                    result[ni, nj] = strong
                    stack.append((ni, nj))

# 清除孤立弱邊緣

```

蹤

```

    result[result  $\neq$  strong] = 0
    return result

# Canny邊緣檢測
# 灰階 → 高斯平滑 → Sobel梯度 → 非極大值抑制 → 雙閾值與邊緣連接
def my_canny(path):
    # 1. 讀取影像
    img = cv2.imread(path)
    # 2. 轉灰階
    gray = rgb2gray(img)
    # 3. 高斯模糊
    blur = gaussian_blur(gray, size=3, sigma=1.4)
    # 4. Sobel梯度計算
    mag, dir = sobel_gradient(blur)
    # 5. 非極大值抑制
    nms = non_max_suppression(mag, dir)
    # 6. 雙閾值與邊緣連接
    final = hysteresis(nms, low=30, high=90)
    # 與內建canny對照
    cv_canny = cv2.Canny(gray, 30, 90)

    # 回傳各階段結果
    return gray, blur, mag, nms, final, cv_canny

# 6. Hough Transform
# 定義機率式霍夫直線轉換
# def my_hough_lines_p(canny邊緣圖, rho解析度, theta解析度, 閾值, 最小線段長度, 最大線段間隙):
# 霍夫累加器投票 ⇒ 找出直線 ⇒ 提取線段
def my_hough_lines_p(edge_image, rho_res=1, theta_res=1, threshold=100, minLineLength=100, maxLineGap=10):
    h, w = edge_image.shape

    # 計算最大可能的ρ範圍(ρ最大不會超過對角線長度)
    diag_len = int(np.ceil(np.sqrt(h**2 + w**2)))
    # 建立ρ所有可能值
    rhos = np.arange(-diag_len, diag_len + 1, rho_res)

```

```

# 角度從0到180度 並轉成弧度
thetas = np.deg2rad(np.arange(0, 180, theta_res))
# 建立霍夫累加器 ( $\rho$ ,  $\theta$ )投票空間
accumulator = np.zeros((len(rhos), len(thetas)), dtype=np.uint32)
# 找出所有邊緣點座標  $\Rightarrow$  只對邊緣點投票
y_idx, x_idx = np.where(edge_image > 0)

# 計算 $\cos\theta$ 與 $\sin\theta$ 值，避免重複計算
cos_t = np.cos(thetas)
sin_t = np.sin(thetas)

# 投票核心
# 對每個邊緣點計算所有可能的 $\rho$ 值，並在累加器中對應位置投票
for t_idx in range(len(thetas)):
    # 計算該邊緣點對應的 $\rho$ 值
    rho_values = x_idx * cos_t[t_idx] + y_idx * sin_t[t_idx]
    # 轉成累加器的 $\rho$ 索引
    rho_indices = np.round(rho_values).astype(np.int32) + diag_len
    # 避免索引超出範圍
    valid_mask = (rho_indices  $\geq$  0) & (rho_indices < len(rhos))
    # 在累加器中對應位置投票
    np.add.at(accumulator[:, t_idx], rho_indices[valid_mask], 1)

# 找出累加器中投票數大於閾值的位置
result_indices = np.argwhere(accumulator  $\geq$  threshold)
# 沒有偵測到直線就直接結束
if len(result_indices) == 0:
    return []

# 取得每條線的投票數
votes = accumulator[result_indices[:, 0], result_indices[:, 1]]
# 根據投票數排序結果(由大到小)
sort_idx = np.argsort(-votes)
result_indices = result_indices[sort_idx]

# 提取線段
lines_p = []
# 將邊緣點座標組合成陣列

```



```

edge_points = np.column_stack((x_idx, y_idx))

# 對每條偵測到的線進行線段提取
for r_idx, t_idx in result_indices[:300]:
    # 取得ρ與θ值
    rho = rhos[r_idx]
    theta = thetas[t_idx]

    # 計算直線的法向量 (a, b)
    a = np.cos(theta)
    b = np.sin(theta)
    # 設定距離容差範圍
    tolerance = 1.5

    # 計算所有邊緣點到直線的距離
    distances = np.abs(edge_points[:, 0] * a + edge_points[:, 1] *
b - rho)
    # 找出距離在容差範圍內的點
    on_line = distances < tolerance

    # 如果該線上點數少於2，則跳過
    if np.sum(on_line) < 2:
        continue
    # 取得該線上的邊緣點
    line_points = edge_points[on_line]

    # 投影成一維排序 ⇒ 判斷線段連續性
    projected = line_points[:, 0] * (-b) + line_points[:, 1] * a
    # 依線段方向排序
    order = np.argsort(projected)
    # 同步排序點
    projected = projected[order]
    line_points = line_points[order]

    # 分割線段
    segments = []
    start_i = 0

```

```

# 檢查相鄰點距離
for i in range(len(projected) - 1):
    # 計算斷裂距離
    gap = projected[i + 1] - projected[i]
    # 如果距離大於最大允許間隙，則分割線段
    if gap > maxLineGap:
        # 計算線段長度
        length = projected[i] - projected[start_i]
        # 如果線段長度大於最小線段長度，則保存該線段
        if length ≥ minLineLength:
            # 線段起點和終點
            p_start = line_points[start_i]
            p_end = line_points[i]
            # 保存線段 (x1, y1, x2, y2)
            segments.append((p_start[0], p_start[1], p_end[0],
p_end[1]))

        # 重新開始新線段
        start_i = i + 1

# 處理最後一段線段
if start_i < len(projected):
    # 計算線段長度
    length = projected[-1] - projected[start_i]
    # 如果線段長度大於最小線段長度，則保存該線段
    if length ≥ minLineLength:
        p_start = line_points[start_i]
        p_end = line_points[-1]
        segments.append((p_start[0], p_start[1], p_end[0],
p_end[1]))

    # 將該直線的所有線段加入結果
    lines_p.extend(segments)

return lines_p

# 加入浮水印
def add_watermark(img, sign_path):

```

```

# 讀取浮水印圖案
# IMREAD_UNCHANGED ⇒ 包含alpha通道(透明通道)
sign = cv2.imread( sign_path, cv2.IMREAD_UNCHANGED)

# 分離RGB與Alpha通道
if sign.ndim == 3 and sign.shape[2] == 4:
    b, g, r, a = cv2.split(sign)
    # 存顏色
    sign_rgb = cv2.merge((b, g, r))
    # 存透明度
    alpha = a
# 處理沒有透明通道的浮水印
# 灰階 轉成 BGR + 完全不透明
else:
    sign_rgb = sign if sign.ndim == 3 else cv2.cvtColor(sign,
cv2.COLOR_GRAY2BGR)
    alpha = np.ones(sign_rgb.shape[:2], dtype=np.uint8) * 255

# 計算浮水印大小 (寬度為原影像寬度的1/5)
h_img, w_img = img.shape[:2]
max_width = max(1, w_img // 5)
# 防止浮水印寬度為0
if sign_rgb.shape[1] == 0:
    return img

# 計算新浮水印大小，等比例縮放
scale = max_width / sign_rgb.shape[1]
new_w = max(1, int(sign_rgb.shape[1] * scale))
new_h = max(1, int(sign_rgb.shape[0] * scale))
sign_rgb = cv2.resize(sign_rgb, (new_w, new_h),
interpolation=cv2.INTER_AREA)
alpha = cv2.resize(alpha, (new_w, new_h),
interpolation=cv2.INTER_AREA)

# 如果原圖是灰階，先轉成BGR
if img.ndim == 2:
    dst = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)

```

```

        # was_gray 標記原圖為灰階
        was_gray = True
    else:
        dst = img.copy()
        was_gray = False

    # 計算浮水印放置位置（右下角）
    x_offset = w_img - new_w - 10
    y_offset = h_img - new_h - 10
    if x_offset < 0: x_offset = 0
    if y_offset < 0: y_offset = 0

    # 正規化alpha通道到0-1之間
    alpha_f = (alpha.astype(np.float32) / 255.0)[ ... , None] # shape
(h,w,1)

    # 混合浮水印與原圖
    for y in range(new_h):
        for x in range(new_w):
            # 透明度比例
            a = alpha_f[y, x, 0]
            if a ≤ 0:
                continue
            src_px = sign_rgb[y, x].astype(np.float32)
            dst_px = dst[y + y_offset, x + x_offset].astype(np.float32)
            # 進行alpha混合
            blended = (a * src_px + (1 - a) * dst_px).astype(np.uint8)
            dst[y + y_offset, x + x_offset] = blended

    # 如果原圖是灰階，轉回灰階
    if was_gray:
        return cv2.cvtColor(dst, cv2.COLOR_BGR2GRAY)
    return dst

file_list = ['1.jpg', '2.jpg', '3.jpg']

```

```

for filename in file_list:
    img = cv2.imread(filename)
    # 執行Canny邊緣檢測
    gray, blur, mag, nms, my_canny_result, true_canny =
my_canny(filename)
    # 複製原圖，準備畫線段
    output_custom = img.copy()

    # 執行Hough Transform
    my_lines = my_hough_lines_p(my_canny_result, rho_res=1,
theta_res=1, threshold=100, minLineLength=100, maxLineGap=10)

    # 繪製偵測到的線段
    for x1, y1, x2, y2 in my_lines:
        cv2.line(output_custom, (int(x1), int(y1)), (int(x2), int(y2)),
(0, 255, 0), 2)

    # 加入浮水印
    canny_with_sig = add_watermark(my_canny_result.copy(),
sign_path='white_signed.png')
    hough_lines_p_with_sig = add_watermark(output_custom.copy(),
sign_path='signed.png')

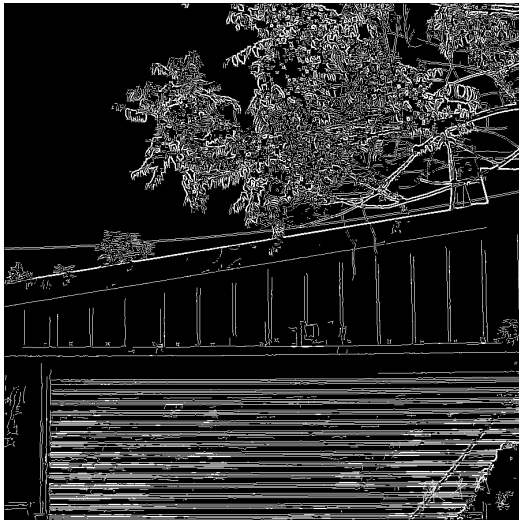
    # 取得檔名與副檔名
    name, ext = os.path.splitext(filename)

    # 顯示和儲存結果
    cv2.imshow(f"canny - {name}", canny_with_sig)
    cv2.imshow(f"Hough_LinesP_My_Result- {name}",
hough_lines_p_with_sig)
    cv2.imwrite(f"{name}_Canny_Edge_Detection{ext}", canny_with_sig)
    cv2.imwrite(f"{name}_Hough_Transform{ext}", hough_lines_p_with_sig)

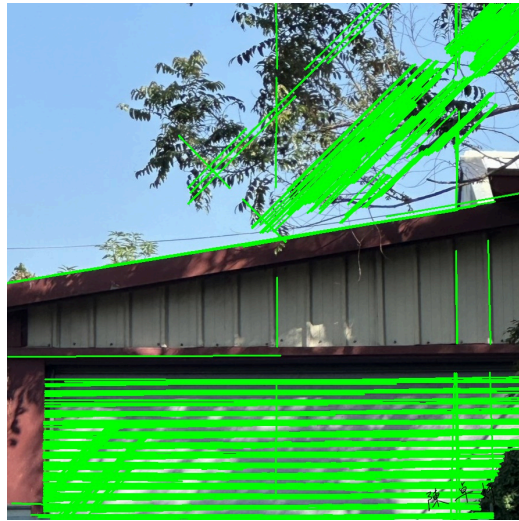
    cv2.waitKey(0)
    cv2.destroyAllWindows()

```

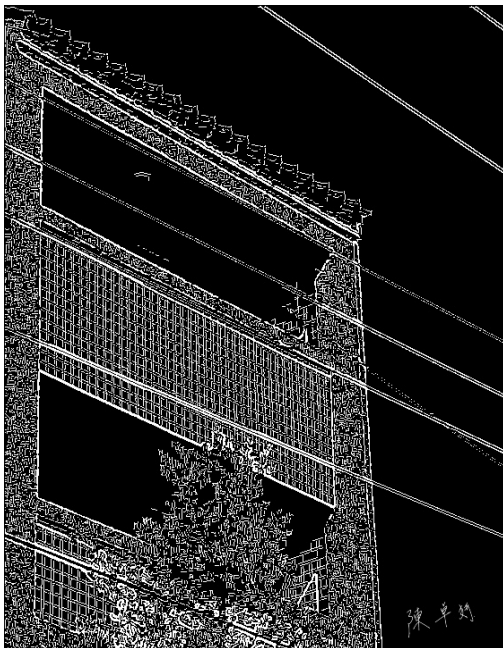
三、輸出結果



圖四、1_Canny Edge Detection. jpg



圖五、1_Hough_Transform. jpg



圖六、2_Canny Edge Detection. jpg



圖七、2_Hough_Transform. jpg



圖八、3_Canny_Edge_Detection. jpg



圖九、3_Hough_Transform. jpg

四、心得

● 不同網格大小 對結果有什麼影響

1. 網格越小 -> 保留更多細節
 - a. 因為計算範圍小，所以對影像的擾動也小
 - b. 邊緣較為清晰
 - c. 細節保留較好
 - d. 但是在邊緣偵測時，可能會出現破碎邊緣。因為小網格太敏感，勿將噪音當作邊緣
2. 中等網格
 - a. 平衡細節與平滑
 - b. 噪音抑制比小網格好
 - c. 適用於一般影像處理
 - d. 但是細微邊緣會開始被模糊 小細節會被消除
3. 大網格
 - a. 保留大輪廓，小細節幾乎消失
 - b. 解析度變差
 - c. 小物體被抹除

● 自製與內建函數比較

1. 執行速度差異：內建比自製快許多，因為自製需要每一個pixel都跑一遍
2. 精準度差異：自製容易出錯、不穩定，結果與使用內建函數不一樣